

Abstain Before You Trust: Risk-Controlled Triage for Automated Vulnerability Detection under Project, Temporal and Dataset Shift

Author names to be completed

Affiliations to be completed

Abstract

Context. Learning-based vulnerability detectors are increasingly proposed as filters that decide which code a human need not inspect, yet they are evaluated by accuracy on random splits, which says little about the share of vulnerabilities such a filter silently lets through. **Objective.** We study when an automated detector can be *trusted to clear code*, by formulating vulnerability triage as a risk-control problem and testing its guarantees under the shifts that characterise software evolution: new projects, later vulnerabilities and different datasets. **Method.** We specialise conformal risk control to the clearance decision (finite-sample miss-rate guarantee, a high-probability variant, group-conditional calibration, a bound on the effect of distribution shift, certified flagging, and a cost-optimal budget) and evaluate it on 487 000 deduplicated C/C++ functions from DiverseVul and Big-Vul in seven scenarios and three seeds, against four detectors and a rule-based analyser. **Results.** Deduplication removes a quarter of Big-Vul and exposes a 43% overlap between the corpora that inflates cross-dataset AUPRC by 3.5–5.6×. Detection quality falls by 43–72% outside the random split. On exchangeable data the guarantee holds (realised miss 9.3% for a 10% budget); under shift it is violated by 2.7× on average (up to 37%), a covariate-shift weighting does not repair it, and about 100 labelled target positives do, at a safe automation rate of 27–32% instead of 54–55%. Miss rates are very uneven across function sizes (41% vs. 2%) and CWEs, and 13.5% of decisions flip under semantics-preserving rewrites. A study with frozen CodeBERT scorers replicates the findings, and reveals a formatting shortcut in the Big-Vul release (a classifier of whitespace and comments alone reaches AUROC 0.92) that makes raw-text models look valid and efficient until the dataset changes (miss

rates up to 48% at a 10% budget). Delegation pays off only when a miss costs tens of reviews. **Conclusion.** Reporting accuracy is not enough: trustworthy automation requires an explicit, monitored miss budget calibrated on the deployment distribution.

Keywords: vulnerability detection, conformal prediction, risk control, distribution shift, trustworthy automation, reproducibility

1. Introduction

Software engineering is rapidly delegating decisions to automated tools: which code needs a security review, which alerts can be ignored, which patch can be merged. Each delegation shifts a question from “is the tool effective?” to “can its output be trusted without inspection?”. For vulnerability detection the answer matters twice: a false alarm wastes reviewer time, but a missed vulnerability is exactly the event that the automation was supposed to prevent, and it is invisible by construction when the tool clears the code.

The research literature on learning-based vulnerability detection reports accuracy, F1 or AUROC, mostly on random splits of public corpora. Such numbers do not answer the question a security team has to answer before it stops reviewing the cleared code: *what fraction of the vulnerable functions will the tool let through, and does that fraction remain valid on the next project, the next release, or another codebase?* Recent studies have shown that accuracy collapses on realistic data and that the corpora themselves are noisy and duplicated Chakraborty et al. (2022); Croft et al. (2023); Ding et al. (2025), but they stop at measuring detection quality. Two parts of the problem remain open: (i) a way to *control* the miss rate of the decisions that are actually delegated, and (ii) an empirical account of whether that control survives the distribution shifts that characterise software evolution.

Approach. We treat clearing a function as a decision that must carry a statistical guarantee, and build it on conformal risk control Vovk et al. (2005); Angelopoulos et al. (2022). The detector is used only as a scorer; a threshold chosen on labelled calibration positives bounds the share of vulnerable functions that are auto-cleared by a user-chosen budget α (Figure 1). The rest of the code goes to human review, or is auto-flagged where a precision guarantee can be certified. We complement the construction with the quantities needed to deploy it: a high-probability version, group-conditional

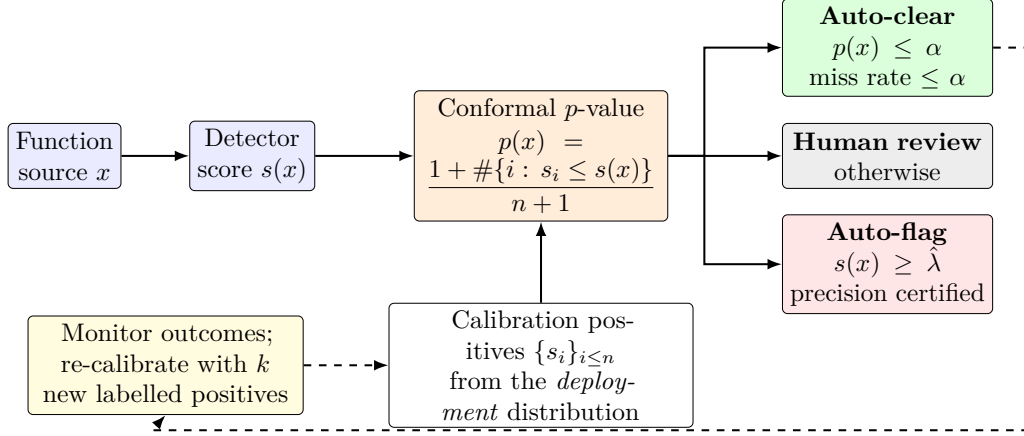


Figure 1: Risk-controlled triage. Calibration positives from the deployment distribution fix the clearance threshold; a function is auto-cleared when its conformal p -value is at most the miss budget α , auto-flagged when a certified precision threshold is met, and reviewed otherwise. Outcomes feed back into re-calibration.

calibration, a bound on how far distribution shift can push the realised miss rate, a cost model for choosing α , and a monitoring/re-calibration loop.

Findings in brief. We test the construction on 487 412 deduplicated C/C++ functions from two public corpora (DiverseVul and Big-Vul), in seven evaluation scenarios (random, project-disjoint, temporal, cross-dataset), with three seeds and four detectors, together with a rule-based analyser. The guarantee is exact on exchangeable data and fails under every kind of shift we examine, by a factor of 2.7 on average; the standard remedy of covariate-shift weighting does not repair it; and a small number of labelled target positives (about 100) does. What can then be automated is about half of what a random-split evaluation suggests. The same holds for scorers built on a pre-trained code model (CodeBERT), and in checking this we found that models reading raw Big-Vul text exploit a formatting shortcut that gives them an apparently valid and efficient filter that fails on other data.

Contributions.

1. **Formulation and theory.** A risk-control formulation of vulnerability triage with operational quantities (miss rate, safe automation rate, review effort) and the following results: validity and calibration-set variability of conformal clearance (Prop. 1, Cor. 1), a high-probability

variant (Prop. 2), a bound on the miss-rate excess under shift in terms of the KS distance (Prop. 3), group-conditional validity (Prop. 4), the identity $\text{AUROC} = \int \text{TNR}(\alpha) d\alpha$ that explains why AUROC is a poor proxy for operational value (Lemma 2), and an economically optimal miss budget (Prop. 5).

2. **A shift-aware empirical test of the guarantee** on seven scenarios, including evidence that source-calibrated clearance misses $2.7\times$ the nominal share under shift, that all 336 observations satisfy the proposed bound, that covariate-shift weighting is insufficient, and how many labelled target positives are needed to recover validity.
3. **A data-quality and leakage audit** of DiverseVul and Big-Vul (24.6% exact duplicates in Big-Vul; 42.7% of Big-Vul inside DiverseVul) and its quantified effect on cross-dataset evaluation.
4. **Accountability analyses:** uneven miss rates across function sizes and CWEs and their repair by Mondrian calibration; decision flips under semantics-preserving rewrites; a TreeSHAP fidelity/consistency study that links the model’s reliance on function size to the unevenness.
5. **Operational evidence:** comparison with a rule-based analyser and a cost analysis that shows when delegation pays off.
6. **A pre-trained-model replication and shortcut audit:** frozen CodeBERT scorers on raw and normalised text, a formatting-only probe that reaches AUROC 0.92 on Big-Vul, and the demonstration that a conformal guarantee can hold for a scorer that relies on such a shortcut.
7. **An open, reproducible artifact** that regenerates every number in this paper from cached scores.

Scope. Because the study had to be completed on CPU-only infrastructure, the main detectors are TF-IDF/gradient-boosting models, complemented by a subsample experiment with frozen (not fine-tuned) CodeBERT embeddings (Section 5.6). The triage layer is detector-agnostic and the theoretical conclusions do not depend on the scorer, but the numerical levels (for example the achievable SAR) will differ for stronger models; Section 7 discusses this.

Structure. Section 2 places the work in the literature, Section 3 develops the theory, Section 4 describes the study, Section 5 answers the five research questions, and Sections 6–8 discuss implications, threats and conclusions.

2. Background and related work

Learning-based vulnerability detection and its evaluation. Function-level detectors based on graph networks Zhou et al. (2019), token models and pre-trained code language models are commonly reported with high accuracy on random splits of public corpora. Chakraborty et al. Chakraborty et al. (2022) showed that performance drops sharply on realistic data, and recent studies with PrimeVul Ding et al. (2025) reach similar conclusions for code language models under de-duplicated, time-aware evaluation. The data underneath is itself a threat: label noise and duplication in vulnerability corpora are documented Croft et al. (2023), and code duplication inflates the results of machine-learning models of code Allamanis (2019). The same concern about space–time bias is well established in malware detection Pendlebury et al. (2019). Our audit (Section 4) and the leakage experiment (Section 5.1) quantify these issues for the two corpora we use and carry the analysis one step further, from how *accurate* a detector is to what a team can *safely delegate* to it.

Calibration, selective prediction and uncertainty. Neural networks are often miscalibrated Guo et al. (2017) and post-hoc scaling and isotonic regression are standard remedies. Selective classification lets a model abstain on uncertain inputs and trades coverage for risk Geifman and El-Yaniv (2017). Uncertainty estimation has begun to appear in vulnerability detection, for example Bayesian recurrent detectors and toolkits that quantify the uncertainty of code language models. These approaches provide *scores*, not statistical guarantees on the error of the delegated decisions, and calibration alone does not give a guarantee on a specified class-conditional error such as the share of missed vulnerabilities.

Conformal prediction and risk control. Conformal prediction Vovk et al. (2005); Shafer and Vovk (2008); Angelopoulos and Bates (2021) converts any scorer into a procedure with finite-sample, distribution-free coverage under exchangeability; conformal risk control Angelopoulos et al. (2022) and Learn-then-Test Angelopoulos et al. (2021) generalise it to controlling expected loss and to high-probability statements about arbitrary risks; weighted conformal prediction Tibshirani et al. (2019) extends validity to covariate shift with a known likelihood ratio. Class-conditional miss-rate control of this kind is used for safety warning systems in robotics Luo et al. (2022). To the best of our knowledge, and based on our literature search, the application of

such guarantees to *automated vulnerability triage*, together with an empirical analysis of how the guarantee behaves under the project, temporal and cross-dataset shifts that characterise software evolution, has not been studied before. Our contribution is not a new conformal theory, but (i) its specialisation to the clearance decision of a vulnerability detector, with a diagnostic bound on the effect of shift (Proposition 3), the high-probability variant, and an economic rule for choosing the budget; and (ii) a large empirical test of when these guarantees hold in practice.

Static analysis and alert fatigue. Rule-based analysers produce many alerts that developers ignore Johnson et al. (2013). We include Flawfinder as a representative lightweight analyser and compare it with the learned triage layer on identical functions.

Robustness and explanation of code models. Models of code are sensitive to semantics-preserving transformations such as identifier renaming Rabin et al. (2021); Yefet et al. (2020). For trustworthy automation what matters is whether the *delegated decision* changes, so we measure decision flips of the triage layer rather than only score changes. For explanation we use TreeSHAP Lundberg and Lee (2017); Lundberg et al. (2020) on an interpretable metric-based model, evaluate fidelity by deletion, and test consistency of the explanations across seeds and across datasets.

3. Risk-controlled triage: formulation and guarantees

This section formalises the triage layer and states the guarantees that the empirical study (Section 4) tests. All proofs are short and are given in full so that the assumptions under which each guarantee holds, and therefore the conditions under which it fails, are explicit.

3.1. Setting and operational quantities

Let X be a function (source text) and $Y \in \{0, 1\}$ indicate that it is vulnerable, with prevalence $\pi = \Pr(Y = 1)$. A trained detector yields a real-valued *score* $S = s(X)$, where larger values indicate higher vulnerability likelihood; s may be any monotone proxy of a probability (a logit, a margin, or a calibrated probability). For a distribution P over (X, Y) and a threshold τ we define

$$M_P(\tau) = \Pr_P(S \leq \tau \mid Y = 1), \quad (1)$$

$$\text{TNR}_P(\tau) = \Pr_P(S \leq \tau \mid Y = 0), \quad (2)$$

$$\text{SAR}_P(\tau) = \Pr_P(S \leq \tau). \quad (3)$$

Here M is the *miss rate* (the share of vulnerable functions that are auto-cleared), TNR the share of benign functions that are auto-cleared, and SAR the *safe automation rate*, the share of all functions that are auto-cleared.

A triage policy with clearance threshold τ_c and flag threshold $\tau_f \geq \tau_c$ assigns every function to one of three outcomes: *clear* if $S \leq \tau_c$, *flag* if $S \geq \tau_f$, and *review* otherwise. Only the clear decision removes a function from human attention without evidence, so it is the decision that must carry a safety guarantee; flagging consumes (or, with certification, saves) reviewer time but does not hide vulnerabilities.

Lemma 1 (Decomposition of automation). $\text{SAR}(\tau) = (1 - \pi) \text{TNR}(\tau) + \pi M(\tau)$. When nothing is auto-flagged, the expected number of functions a team must review per 1 000 is $\text{ERE}(\tau) = 1000 (1 - \text{SAR}(\tau))$.

Proof. Condition on Y in the law of total probability: $\Pr(S \leq \tau) = \Pr(S \leq \tau \mid Y=0)(1 - \pi) + \Pr(S \leq \tau \mid Y=1)\pi$. $\square$

Because $\pi \approx 5\%$ in realistic data, Lemma 1 shows that automation is driven almost entirely by TNR at the chosen miss budget $M(\tau) = \alpha$. Write $\text{TNR}(\alpha)$ for the true-negative rate at the threshold whose miss rate is α (the ROC curve reflected in $\alpha = 1 - \text{TPR}$).

Lemma 2 (AUROC is the average clearance potential). If the score distributions are continuous, $\text{AUROC} = \int_0^1 \text{TNR}(\alpha) d\alpha$.

Proof. Let F_1, F_0 be the score CDFs of vulnerable and benign functions, so $\text{TNR}(\alpha) = F_0(F_1^{-1}(\alpha))$. With $U \sim \text{Unif}(0, 1)$, $F_1^{-1}(U)$ has the law of a vulnerable score S_1 ; hence $\int_0^1 F_0(F_1^{-1}(\alpha)) d\alpha = \mathbb{E}[F_0(S_1)] = \Pr(S_0 \leq S_1) = \text{AUROC}$ for an independent benign score S_0 . $\square$

Lemma 2 explains why AUROC is a poor proxy for *operational* value: a safety-relevant budget such as $\alpha = 0.05$ uses only the left tail of the integral, and two detectors with equal AUROC can differ widely in $\text{TNR}(0.05)$. Section 5.1 quantifies this.

3.2. Conformal clearance with a finite-sample miss-rate guarantee

Let $s_1, \dots, s_n$ be the scores of n *vulnerable* functions in a held-out calibration set, and define for a new function x with score s the conformal p -value

$$p(x) = \frac{1 + \#\{i \leq n : s_i \leq s\}}{n + 1}. \quad (4)$$

The policy *clears* x iff $p(x) \leq \alpha$. Equivalently, with $s_{(1)} \leq \dots \leq s_{(n)}$ the order statistics and $k = \lfloor \alpha(n + 1) \rfloor$, it clears iff $s < s_{(k)}$ (and clears nothing if $k = 0$).

Proposition 1 (Finite-sample validity). *If the score S_{n+1} of a vulnerable test function is exchangeable with $s_1, \dots, s_n$, then $\Pr(p(X_{n+1}) \leq \alpha) \leq \alpha$ for every $\alpha \in (0, 1)$; that is, the expected miss rate of the clearance rule is at most α . If scores are almost surely distinct the probability is exactly $\lfloor \alpha(n + 1) \rfloor / (n + 1) \geq \alpha - \frac{1}{n+1}$.*

Proof. Let $R = 1 + \#\{i \leq n : s_i \leq S_{n+1}\}$ so that $p = R/(n + 1)$. R is at least the rank of S_{n+1} among the $n+1$ scores under uniformly random tie-breaking, and that rank is uniform on $\{1, \dots, n+1\}$ by exchangeability. Hence $\Pr(R \leq k) \leq k/(n + 1)$, and $p \leq \alpha \iff R \leq \lfloor \alpha(n + 1) \rfloor$. With distinct scores R is the rank itself, giving equality. $\square$

Corollary 1 (Calibration-set variability and minimum sample size). *For continuous scores, the conditional miss rate given the calibration set, $M(\hat{\tau})$, follows $\text{Beta}(k, n + 1 - k)$ with $k = \lfloor \alpha(n + 1) \rfloor$. Its mean is $k/(n + 1) \leq \alpha$ and its variance is $\approx \alpha(1 - \alpha)/(n + 2)$. A non-trivial rule requires $n \geq \lceil 1/\alpha \rceil - 1$ calibration positives.*

Proof. The rule clears $s < s_{(k)}$, so $M(\hat{\tau}) = F_1(s_{(k)}^-)$, the k -th order statistic of n i.i.d. $\text{Uniform}(0, 1)$ variables after the probability integral transform, which is $\text{Beta}(k, n + 1 - k)$. $\square$

Proposition 1 is a *marginal* (expected) guarantee: roughly half of independent calibration draws will produce a realised miss rate slightly above α (Corollary 1). A practitioner who needs a high-probability statement can use the following variant.

Proposition 2 (PAC clearance). *Fix $\delta \in (0, 1)$ and let $j^* = \max\{j : F_{\text{Beta}(j, n+1-j)}^{-1}(1 - \delta) \leq \alpha\}$. Clearing iff $s < s_{(j^*)}$ satisfies $\Pr_{\text{calibration}}(M(\hat{\tau}) >$*

$\alpha\big) \leq \delta$ for continuous, exchangeable scores (and clears nothing if no such j exists).

Proof. By Corollary 1, $M(\hat{\tau}) \sim \text{Beta}(j^*, n+1-j^*)$, and j^* is chosen so that its $(1 - \delta)$ quantile does not exceed α . $\square$

3.3. What distribution shift does to the guarantee

Exchangeability is the only assumption in Proposition 1, and it is exactly what project-level, temporal and cross-dataset evaluation violate. Let F_c and F_t be the CDFs of the scores of vulnerable functions under the calibration and test distributions, and $D = \sup_s |F_c(s) - F_t(s)|$ their Kolmogorov–Smirnov distance.

Proposition 3 (Shift bound). *For a threshold $\hat{\tau}$ calibrated at level α under P_c with n positives,*

$$\mathbb{E}[M_{P_t}(\hat{\tau})] \leq \alpha + D.$$

The bound is attained when F_t lies uniformly D above F_c at the clearance threshold.

Proof. Pointwise $F_t(\tau) \leq F_c(\tau) + D$ for every τ , hence $M_{P_t}(\hat{\tau}) = F_t(\hat{\tau}^-) \leq F_c(\hat{\tau}^-) + D$. Taking expectations over the calibration draw and applying Proposition 1 to the first term gives the claim. $\square$

Proposition 3 is not a repair but a *diagnostic*: the excess miss rate over the nominal budget cannot exceed the KS distance between the positive-class scores before and after the shift. Figure 5 tests this bound on every shifted scenario. Two remedies are studied.

Covariate-shift weighting. If $\Pr_t(Y|x) = \Pr_c(Y|x)$ and $w(x) = dP_t(x)/dP_c(x)$ is known, the likelihood ratio on the positive class is $w(x) \pi_c/\pi_t$; the constant cancels after normalisation, so the weighted p -value of Tibshirani et al. (2019),

$$p_w(x) = \frac{\sum_{i=1}^n w(x_i) \mathbb{I}[s_i \leq s] + w(x)}{\sum_{i=1}^n w(x_i) + w(x)}, \quad (5)$$

restores validity when computed on the calibration *positives*. We estimate w with a cross-fitted logistic domain classifier on TF-IDF features. The guarantee is only as good as the density-ratio estimate and the covariate-shift assumption, both of which are doubtful for code; we therefore treat it as an empirical question (RQ3).

Recalibration on a small labelled target sample. If k labelled vulnerable functions from the *target* distribution (the new project, or the most recent release window) are available, calibrating on them restores exchangeability by construction, and Proposition 1 applies verbatim with $n = k$. The price is variance: by Corollary 1, the realised miss rate has standard deviation $\approx \sqrt{\alpha(1-\alpha)/k}$, and the PAC variant (Proposition 2) trades part of the clearance rate for a bounded violation probability.

3.4. Group-conditional validity (Mondrian calibration)

The marginal guarantee averages over all vulnerable functions and can hide very uneven risk across inputs that are observable at inference time. Partition the input space by a function $b(x)$ (we use function-length terciles).

Proposition 4 (Mondrian validity). *If clearance in group g uses p -values computed only from calibration positives with $b = g$, then for every group g with exchangeable calibration and test positives, $\Pr(p_g(X) \leq \alpha \mid Y=1, b(X)=g) \leq \alpha$.*

Proof. Condition on $b(X_{n+1}) = g$ and on the set of calibration indices in group g ; scores within the group remain exchangeable, and Proposition 1 applies with n replaced by the group’s calibration size. $\square$

3.5. Certified auto-flagging

Flagging with a guarantee on precision is the dual problem, and harder because it is conditioned on the predicted class. We use the Learn-then-Test fixed-sequence procedure Angelopoulos et al. (2021): half of the calibration set fixes a descending grid of thresholds $\lambda_1 > \lambda_2 > \dots$; on the other half we test, in order, $H_j : \Pr(Y=1 \mid S \geq \lambda_j) < \beta$ with a one-sided Clopper–Pearson bound at level δ , and stop at the first non-rejection. The last rejected threshold $\hat{\lambda}$ then satisfies $\Pr(\Pr(Y=1 \mid S \geq \hat{\lambda}) < \beta) \leq \delta$. Because precision of vulnerability detectors is low (Section 5.3), we expect few or no certifiable flags at practically useful β ; this is itself a finding about what can safely be automated.

3.6. Choosing the miss budget economically

Let one human review cost 1 and one missed vulnerability cost c_m (in review units). The expected cost per function when clearing at miss budget α is

$$C(\alpha) = (1 - \text{SAR}(\alpha)) + c_m \pi \alpha, \quad (6)$$

Algorithm 1 Risk-controlled triage (calibration and deployment)

Require: trained scorer $s(\cdot)$; labelled calibration set $\mathcal{D}_{\text{cal}}$ *from the deployment distribution*; miss budget α ; optional PAC level δ ; optional group map $b(\cdot)$

- 1: $\mathcal{S}^+ \leftarrow \{s(x_i) : (x_i, y_i) \in \mathcal{D}_{\text{cal}}, y_i = 1\}$, sorted; $n \leftarrow |\mathcal{S}^+|$ (per group if Mondrian)
- 2: $k \leftarrow \lfloor \alpha(n+1) \rfloor$ **or** $k \leftarrow j^*(n, \alpha, \delta)$ for the PAC variant; **if** $k = 0$ **then** clear nothing
- 3: $\hat{\tau} \leftarrow s_{(k)}$ (clear iff $s(x) < \hat{\tau}$)
- 4: **for** each new function x **do**
- 5: **if** $s(x) < \hat{\tau}$ **then** auto-clear **else** send to review (or auto-flag if $s(x) \geq \hat{\lambda}$)
- 6: **end for**
- 7: Monitor: on every batch of newly labelled outcomes, re-estimate the miss rate and re-calibrate when it exceeds α

to be compared with $C = 1$ for “review everything” and $C = c_m\pi$ for “review nothing”.

Proposition 5 (Cost-optimal budget). *For differentiable $\text{TNR}(\cdot)$, an interior minimiser of (6) satisfies*

$$\text{TNR}'(\alpha^*) = \frac{\pi(c_m - 1)}{1 - \pi}.$$

Proof. By Lemma 1, $\text{SAR}(\alpha) = (1 - \pi)\text{TNR}(\alpha) + \pi\alpha$, so $C'(\alpha) = -(1 - \pi)\text{TNR}'(\alpha) - \pi + c_m\pi$; set it to zero. $\square$

Since $\text{TNR}'(\alpha)$ is decreasing for concave ROC curves, larger c_m pushes α^* down; if $\text{TNR}'(\alpha) < \pi(c_m - 1)/(1 - \pi)$ already at $\alpha \rightarrow 0$ the optimum is to review everything. Section 5.5 evaluates α^* chosen on calibration data against realised cost on the test data.

3.7. Algorithm

Algorithm 1 summarises calibration and deployment. The calibration step costs one sort of the positive calibration scores; inference adds one binary search per function.

4. Empirical study design

4.1. Research questions

RQ1 (Reliability of detection).

How much does the detection quality of learning-based detectors drop when the evaluation moves from a random split to project-disjoint, temporal and cross-dataset conditions, and is AUROC a faithful predictor of operational value?

RQ2 (Calibration).

How well-calibrated are the probabilities of these detectors, and how much do post-hoc recalibration methods help under shift?

RQ3 (Guarantees under shift).

Does the conformal clearance rule keep its miss-rate budget when calibrated on source data, and which remedies (covariate-shift weighting, labelled target recalibration, the PAC variant) restore it? How many labelled target positives are needed?

RQ4 (Accountability).

Is the guarantee distributed evenly across function sizes and vulnerability classes (CWEs), and does Mondrian calibration repair imbalances? How consistent are the triage decisions and the explanations under semantics-preserving rewrites and across datasets?

RQ5 (Operational value).

How much reviewer effort does risk-controlled triage save at a valid miss budget, how does it compare with a rule-based static analyser, and what does it imply under an explicit cost model?

RQ6 (Pre-trained code model).

Do the conclusions hold for a scorer built on a pre-trained code model (CodeBERT), and do such models exploit dataset artifacts that lexical models cannot see?

4.2. Datasets and data-quality audit

We use two public C/C++ function-level vulnerability datasets that differ in provenance. **DiverseVul** Chen et al. (2023) contains functions collected from security-issue websites and the commits that fix them; each record carries the project, the fixing commit and (where available) the CWE. **BigVul** Fan et al. (2020) links CVE records to fixing commits; we use the pre-fix

Table 1: Dataset audit after normalisation (comment/whitespace stripping). 69,268 normalised functions occur in both datasets: 42.7% of clean Big-Vul and 21.3% of clean DiverseVul.

Dataset	Raw	Exact duplicates	Label-conflict	Clean	Vulnerable	Projects
DiverseVul	328,907	3,025 (0.9%)	1,497	325,138	17,689 (5.4%)	800
Big-Vul	216,568	53,373 (24.6%)	2,333	162,274	8,055 (5.0%)	309

function body as the vulnerable sample, the unchanged functions from the same commits as benign samples, and take the publication year from the CVE identifier as a temporal key. Both were obtained from the Hugging Face hub (`claudios/DiverseVul`, `bstee615/bigvul`). We deliberately did not use the Juliet test suite, whose synthetic flaws make random-split results saturate, nor PrimeVul, which was not retrievable in our environment.

Pre-processing and audit. For each function we strip comments, collapse whitespace and hash the result. Within a dataset we remove exact duplicates, and we drop every hash that occurs with conflicting labels. Across datasets we compute the overlap of normalised hashes. Table 1 reports the audit; the consequences for evaluation are discussed in Section 5.1.

4.3. Evaluation scenarios

Table 2 defines seven scenarios. In every scenario the detector is trained on the *train* part; the *ID calibration* part is a random hold-out of the training distribution (what a practitioner would naturally use); the *test* part is where all reported numbers are measured; and, for the shifted scenarios, a labelled *target pool* drawn from the deployment distribution is available for the recalibration experiments but disjoint from the test part.

- **Random** (DV-R, BV-R): 70/15/15 split of deduplicated functions; calibration and test are exchangeable.
- **Project** (DV-P, BV-P): 70% of projects are used for training and calibration; the remaining projects form the test set (a new codebase is deployed). A random 25% of the held-out projects’ functions is the target pool.
- **Temporal** (BV-T): train and ID calibration on CVEs published up to 2015, target pool = 2016, test = 2017–2019 (a model deployed on newer vulnerabilities).

Table 2: Evaluation scenarios (seed 0 sizes; training keeps all vulnerable and at most 150 000 benign functions).

Scenario	Shift	Train	ID cal.	Target pool	Test	Test pos.
DV random	random 70/15/15	162,475	48,879	–	48,553	2,637
BV random	random 70/15/15	113,482	24,489	–	24,303	1,254
DV project	held-out projects (30%)	159,829	30,432	31,053	91,423	4,522
BV project	held-out projects (30%)	124,551	22,227	3,826	11,670	699
BV temporal	train ≤ 2015 ; test 2017–19	60,101	10,850	27,022	52,669	2,288
DV→BV	train DV, test BV (leak-free)	165,052	48,553	25,869	77,380	4,240
BV→DV	train BV, test DV (leak-free)	137,971	24,303	66,735	199,348	11,207

- **Cross-dataset** (DV→BV, BV→DV): train on 85% of one dataset, calibrate on the remaining 15%, test on the other dataset after *removing every test function whose normalised hash occurs in the training data*; a random 25% of the remaining target data is the pool. Removed (leaked) functions are evaluated separately to measure how much leakage would have inflated the result.

Each scenario is repeated with three seeds (different partitions and model initialisations).

4.4. Detectors

Because the experiments must be fully reproducible on commodity CPUs and over 20 scenario–seed pairs, we use four lightweight, widely used detectors on the full corpora, complemented by a frozen-CodeBERT experiment on a subsample (Section 4.5); the absence of fine-tuned code models is a limitation discussed in Section 7, and the triage layer is detector-agnostic.

- **TF-IDF LR** and **TF-IDF SVM**: unigram+bigram code tokens hashed to 2^{19} features, sublinear TF-IDF, logistic regression ($C=20$) or linear SVM ($C=0.3$).
- **LightGBM** Ke et al. (2017): gradient-boosted trees on 71 engineered code metrics (size, nesting depth, cyclomatic proxy, pointer/cast/bit-operation counts, counts of 32 security-relevant API calls) plus the 20 000 most frequent hashed tokens; early stopping on 10% of the training data.
- **LR+LGBM**: sum of standardised LR margin and LightGBM logit.

Training sets keep all vulnerable functions and at most 150 000 benign ones; this does not affect the triage layer, which depends on score ranks only. The rule-based baseline is **Flawfinder** 2.0.20, with the highest risk level among a function’s hits (then the number of hits) as score.

4.5. Pre-trained code model experiment (RQ6)

To test whether the findings depend on the lightweight detectors, we add scorers built on CodeBERT Feng et al. (2020) (`microsoft/codebert-base`). Because the study ran without a GPU, the encoder is *frozen*: we embed the first 256 tokens of each function and use the concatenation of the [CLS] vector and the mean-pooled last layer (1 536 dimensions) as features of a standardised logistic regression ($C=0.05$) and of a LightGBM (embedding plus the 71 code metrics); their standardised sum is the “CodeBERT ensemble”. Embedding the full corpora (487 000 functions) is infeasible on CPU, so we use a stratified subsample of each corpus with all scenarios, splits and seeds unchanged: 5 000 vulnerable and 10 000 benign functions per dataset (30 000 in total; the benign functions are a uniform sample, so positive-class quantities such as the miss rate are unaffected, while AUPRC and SAR are weighted back to the true prevalence). All scorers in this experiment, including the TF-IDF models, are retrained on the subsample so that comparisons are matched. Two input variants are evaluated: *raw text*, and *normalised text* (comments removed, whitespace collapsed), the same normalisation used in the main study. A *formatting-only* LightGBM on 15 formatting statistics of the raw text (indentation, trailing whitespace, comment markers, line lengths) serves as a shortcut probe. For cross-dataset scenarios this experiment is stricter than the main study: any target function that occurs anywhere in the full source corpus is removed.

4.6. Metrics

Detection: AUROC, AUPRC, and F1, MCC, precision, recall at the F1-optimal threshold chosen on ID calibration data. **Calibration:** expected calibration error with 15 equal-mass bins, $ECE = \sum_b \frac{|B_b|}{n} |\bar{y}_b - \bar{p}_b|$, and the Brier score, for raw probabilities, Platt scaling and isotonic regression fitted on ID calibration data. **Triage:** realised miss rate M on test positives against the nominal $\alpha \in \{0.02, 0.05, 0.10, 0.20\}$, the share of cleared functions (SAR, Eq. (3)), the review effort ERE, and the violation frequency $\Pr(M > \alpha + 0.02)$ across 200 random calibration draws of 300 positives (60 for the weighted variant); the 2-point tolerance reflects the sampling error of the

test-set miss rate. For certified flagging we report the flagged share and the realised precision. **Accountability:** miss rate per length tercile and per CWE. **Robustness:** the share of triage decisions that change after identifier renaming, dead-code insertion, or both, and the miss rate on rewritten inputs. **Explanation:** TreeSHAP Lundberg et al. (2020) deletion fidelity, rank stability and cross-dataset consistency of feature importances. **Operational:** expected cost (6) with the budget α^* chosen on calibration data.

4.7. Statistical protocol

Results are reported as means over the three seeds with standard deviations. Detectors are compared with the Friedman test over the 21 scenario-seed blocks; paired comparisons with the best detector use the Wilcoxon signed-rank test and Cliff’s δ as effect size. Whether the realised miss rate exceeds its nominal level is tested with a one-sided Wilcoxon test on the per-block differences between realised and nominal miss rate. Relations between quantities (AUROC versus clearable share, KS distance versus excess miss rate) are measured with Spearman’s ρ . All thresholds, hyper-parameters and splits are fixed before looking at test data; the 2-point tolerance and the budgets α are the only analysis choices and are reported in full.

4.8. Implementation and availability

All code (data preparation, feature extraction, models, triage layer, experiments, figure/table generation and this manuscript) is in the repository accompanying the paper, with a unit test that checks the clearance rule against its p -value definition and its finite-sample validity by simulation. Every number in the paper is produced by the scripts from cached detector scores.

5. Results

5.1. RQ1: Detection quality collapses outside the random split, and AUROC hides what matters

Data audit and leakage. The audit (Table 1) shows that the corpora are less clean than their raw size suggests: 24.6% of the Big-Vul rows (53 373) are exact duplicates after normalisation, against 0.9% for DiverseVul, and 69 268 normalised functions occur in both corpora, i.e. 42.7% of clean Big-Vul and 21.3% of clean DiverseVul. In the cross-dataset scenarios, about 44 000 functions of the evaluated target test part (more than a quarter of Big-Vul,

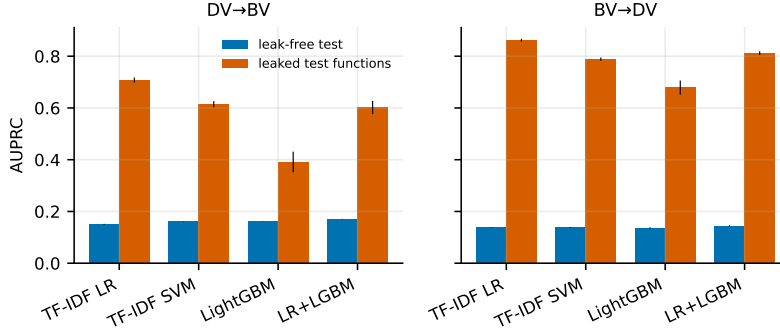


Figure 2: AUPRC on leak-free versus leaked cross-dataset test functions (mean $\pm$ s.d. over 3 seeds). The prevalence of vulnerable functions is about 5%.

Table 3: Effect of train–test leakage in the cross-dataset setting: “leaked” test functions have a normalised hash that occurs in the training data.

Scenario	Model	Leaked n	AUPRC clean	AUPRC leaked	AUROC clean	AUROC leaked
DV→BV	TF-IDF LR	44,180	0.151 $\pm$ 0.002	0.708 $\pm$ 0.010	0.679 $\pm$ 0.001	0.984 $\pm$ 0.002
DV→BV	LightGBM	44,180	0.163 $\pm$ 0.000	0.392 $\pm$ 0.040	0.711 $\pm$ 0.003	0.877 $\pm$ 0.009
BV→DV	TF-IDF LR	44,120	0.137 $\pm$ 0.001	0.862 $\pm$ 0.005	0.711 $\pm$ 0.001	0.951 $\pm$ 0.003
BV→DV	LightGBM	44,120	0.136 $\pm$ 0.002	0.679 $\pm$ 0.027	0.709 $\pm$ 0.006	0.909 $\pm$ 0.007

and about 13% of DiverseVul) also appear in the training data. Figure 2 and Table 3 show what this does to a naive cross-dataset evaluation: for the LR+LGBM scorer, AUPRC is 0.171 on leak-free Big-Vul functions but 0.602 on the leaked ones ($3.5\times$), and 0.144 vs. 0.812 ($5.6\times$) when training on Big-Vul and testing on DiverseVul. A cross-dataset result that does not remove shared functions would therefore report a performance that is between three and a half and almost six times better than what a new codebase would experience.

Performance under shift. Table 4 and Figure 3 give the detection results. On random splits the best scorer reaches AUPRC 0.248 (DiverseVul) and 0.407 (Big-Vul), with AUROC 0.836 and 0.869. When whole projects are held out, AUPRC falls to 0.141 (-43%) and 0.151 (-63%); the temporal split on Big-Vul yields 0.113 (-72%), and the leak-free cross-dataset settings give 0.171 and 0.144. These values are only two to three times the prevalence ($\approx 5\%$), and MCC drops to 0.12–0.17 for the best model (Table 4, panel C), against 0.29 and 0.39 on random splits. The ID-calibration AUPRC (what a practitioner would see on a hold-out of the training distribution)

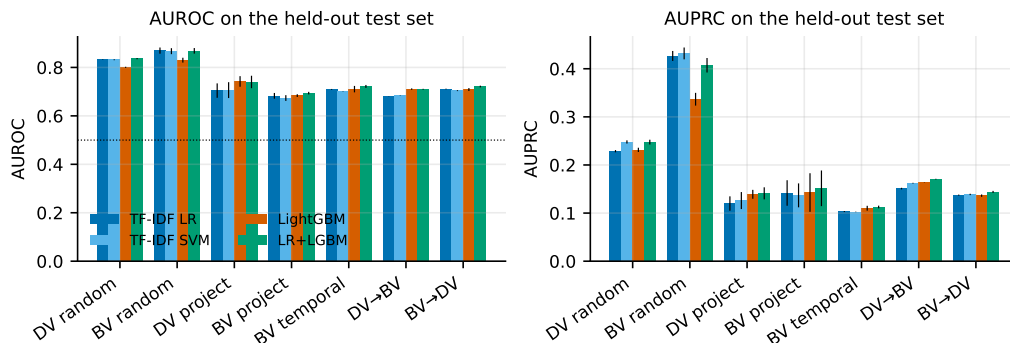


Figure 3: Detection performance of the four scorers across the seven scenarios (mean $\pm$ s.d. over 3 seeds).

is 0.24–0.43 in every scenario, so the optimism of in-distribution validation reaches 1.5–3.8 $\times$ for project, temporal and cross-dataset deployment (0.264 vs. 0.141 for DiverseVul project, 0.425 vs. 0.113 for Big-Vul temporal). The Friedman test over the 21 scenario–seed blocks rejects equal AUPRC for the four scorers ($\chi^2 = 21.9$, $p < 10^{-4}$); the LR+LGBM scorer has the best mean rank (1.48), and beats TF-IDF LR ($p = 0.011$, Cliff’s $\delta = 0.18$) and LightGBM ($p < 10^{-5}$, $\delta = 0.14$), while the difference to the SVM is borderline ($p = 0.055$, $\delta = 0.14$). Differences between scorers are small relative to the drop caused by the evaluation setting. We therefore use LR+LGBM as the scorer in the triage experiments unless stated otherwise.

AUROC is a poor proxy for operational value. By Lemma 2, AUROC averages the clearable fraction of benign functions over all miss budgets. A team that wants at most 5% of vulnerabilities cleared by the tool uses only the left tail. Across the 84 scorer–scenario–seed combinations, the Spearman correlation between AUROC and the share of functions that can be cleared at $\alpha = 5\%$ is only $\rho = 0.38$. Within a scenario, scorers whose mean AUROC differs by 0.02–0.04 differ by 0.04–0.13 in clearable share (e.g. DiverseVul random: AUROC range 0.036, SAR range 0.114). Model selection for triage must therefore use the operating point of interest, not a global ranking metric.

A formatting shortcut in the Big-Vul release. While preparing the pre-trained-model experiment (Section 5.6) we found that the raw text of the Hugging Face release of Big-Vul carries a strong formatting signal: a classifier that sees only 15 formatting statistics (indentation, trailing whitespace, comment markers,

line lengths; Table 15) reaches AUROC 0.919 on a random split, higher than every semantic model in this paper. All detectors in Sections 5.1–5.5 operate on comment-stripped, whitespace-insensitive tokens and code metrics, and removing the only size-sensitive metrics (characters, lines, tokens) changes the AUROC of the LightGBM scorer by at most 0.002 (0.820 $\rightarrow$ 0.818 on Big-Vul, 0.801 $\rightarrow$ 0.802 on DiverseVul), so the main results are not driven by it; but results obtained by models that read raw Big-Vul text should be interpreted with this in mind.

5.2. RQ2: Calibration is acceptable in-distribution and degrades under shift

Table 5 reports ECE. Raw probabilities are mildly miscalibrated even on random splits (LR: 0.015–0.032; LightGBM: 0.0075–0.018), largely because training keeps at most 150 000 benign functions. Platt scaling and isotonic regression fitted on ID calibration data reduce ECE to 0.004–0.015 on random splits, but under shift the residual error is much larger: 0.015–0.023 for Big-Vul project, temporal and both cross-dataset settings, i.e. 30–46% of the 5% base rate, and the two methods perform alike. The one exception is LightGBM trained on DiverseVul and tested on Big-Vul, where raw probabilities (0.008) beat recalibrated ones (0.018). Importantly, because the clearance rule depends only on the ranking of scores, post-hoc calibration changes *no* triage decision; calibration matters for reporting probabilities and for cost-based decisions, not for the miss-rate guarantee.

5.3. RQ3: The miss-rate guarantee holds under exchangeability and fails under shift

Validity on exchangeable splits. Figure 4 (left) and Table 6 show that on random splits the realised miss rate matches the nominal budget: at $\alpha = 10\%$ it is 10.1% (DiverseVul) and 8.6% (Big-Vul); over all budgets and seeds the excess over α is -0.4 percentage points on average (one-sided Wilcoxon $p = 0.80$ for “exceeds”), and the realised-to-nominal ratio is 0.96. This confirms Proposition 1 on real detector scores. At this budget the rule clears 54–55% of all functions (SAR; Figure 7), i.e. it reduces the number of functions that must be reviewed per 1 000 from 1000 to about 450.

Failure under shift. In every shifted scenario the guarantee is violated (Figure 4, right). At $\alpha = 10\%$ the realised miss rate is 24.0% (Big-Vul project), 23.6% (DiverseVul project), 21.7% (Big-Vul temporal), 20.3% (Big-Vul $\rightarrow$ DiverseVul) and 37.0% (DiverseVul $\rightarrow$ Big-Vul), on average $2.7\times$ the budget (excess +12.8

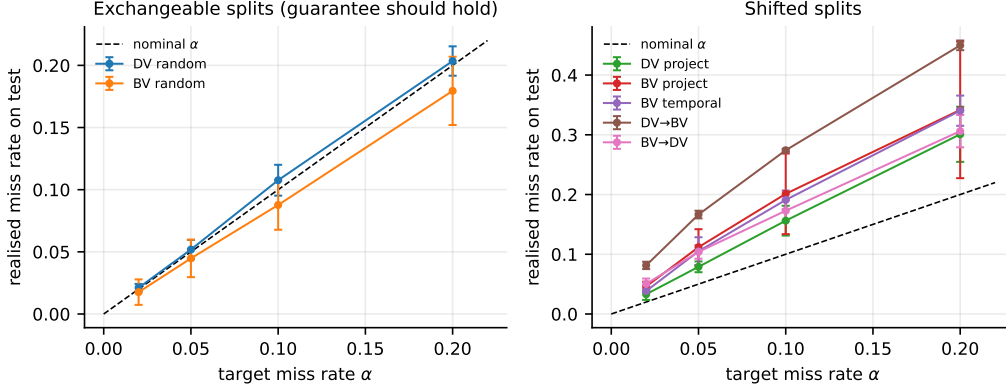


Figure 4: Realised versus nominal miss rate for source-calibrated conformal clearance (LightGBM scorer; mean $\pm$ s.d. over seeds). The dashed line is the target.

points, $p < 10^{-11}$). At $\alpha = 2\%$ the miss rates are 5.0–10.7%. The severity of the project-level violation depends on which projects are held out: for Big-Vul project the realised miss rate at $\alpha = 10\%$ is 16.9, 32.0 and 23.0% in the three partitions (and 9.4, 17.7 and 3.0% in the three partitions of the subsample used in Section 5.6, where it can even fall below the nominal level), whereas the temporal and cross-dataset violations are stable across seeds (20.1–24.2%, 18.3–22.0%, and 36.7–37.2%). Project shift is thus a high-variance threat, temporal and dataset shift a consistent one. Meanwhile the SAR reported for the invalid rule is deceptively high (47.6–66.1%, Table 7), so a team calibrating on a convenient hold-out would believe the tool is both safe and efficient. Proposition 3 bounds the excess by the KS distance between the scores of calibration and test positives; Figure 5 shows that all 336 (scenario, seed, scorer, α) points lie below the bound. The excess is strongly related to the shift ($\rho = 0.74$). The bound is conservative in typical cases (the KS distance is a median $3.1\times$ the excess, inter-quartile range 1.9–6.2) but nearly tight in the worst one (excess up to 95% of the KS distance), so it is a safe diagnostic, usable only if some labelled target positives exist.

Remedies. Table 7 and Figure 6 compare the remedies at $\alpha = 10\%$. *Covariate-shift weighting* helps in two scenarios (DiverseVul project: 23.6 $\rightarrow$ 15.1%; DiverseVul $\rightarrow$ Big-Vul: 37.0 $\rightarrow$ 14.9%) but not in the other three (Big-Vul project 24.0 $\rightarrow$ 18.6%, Big-Vul temporal 21.7 $\rightarrow$ 22.6%, Big-Vul $\rightarrow$ DiverseVul 20.3 $\rightarrow$ 19.5%); it never restores the budget. The estimated weights are

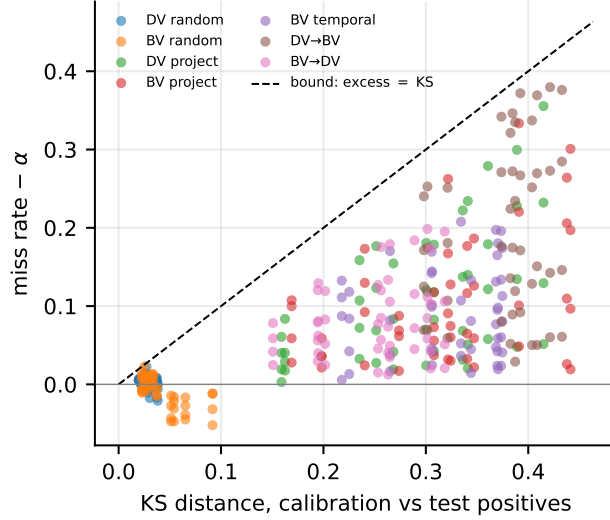


Figure 5: Excess miss rate over the nominal budget versus the KS distance between calibration and test positive scores, for all scorers, budgets and seeds (336 points). All points lie below the diagonal bound of Proposition 3.

extremely skewed: the cap of 20 is reached for 22% of the Big-Vul test functions in DiverseVul→Big-Vul and for 15% in Big-Vul project (median test weight 5.5 and 5.4 against calibration weights around 0.2–0.3), so a few calibration positives carry most of the weight and the effective calibration size is small; together with a probably violated covariate-shift assumption for code, this explains why weighting is not a reliable repair. *Recalibration on labelled target positives* restores the budget: with $k = 100$ the realised miss rate is 9.2–13.0% across the five shifted scenarios (four of the five scenarios are within $10\% \pm 0.8$ points; Big-Vul project is high at 13.0%), with a SAR of 26.7–31.8%; $k = 50$ already gives 9.2–12.1%, and $k = 25$ is conservative (7.3–9.6%) because the finite-sample correction clears only $\lfloor 0.1 \cdot 26 \rfloor = 2$ order statistics. The empirical curves fall inside the Beta band predicted by Corollary 1. Because the guarantee is in expectation, the frequency with which a single calibration draw exceeds $\alpha + 2$ points is 16–68% at $k = 100$; the PAC variant (Proposition 2) cuts this to $\leq 7\%$ in all scenarios (mostly $\leq 2.3\%$), at the price of a lower SAR (15.9–22.4%). The valid operating points under shift thus clear about 27–32% of functions (non-PAC) or 16–22% (PAC), roughly half and a third of the 54–55% available on random splits. Importantly, the PAC rule calibrated on *source* data remains invalid under

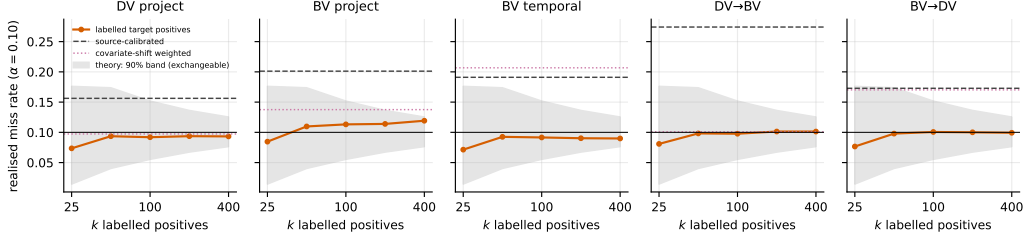


Figure 6: Realised miss rate at $\alpha = 10\%$ as a function of the number k of labelled target positives used for calibration (LightGBM scorer; mean over seeds). The shaded area is the 90% band of Corollary 1 for exchangeable data; dashed and dotted lines are the source-calibrated and weighted rules.

shift (94–100% violations), which confirms that the issue is exchangeability and not the choice of the quantile.

Certified flagging. Table 8 shows that the opposite decision, flagging with certified precision, offers little. With $\beta = 50\%$ the Learn-then-Test procedure certifies a threshold that flags only 0.09% (DiverseVul) and 2.6% (Big-Vul) of the test functions on random splits (realised precision 80% and 57%); for $\beta = 70\%$ the flagged share is below 0.3%. Under shift the certification fails as well: for $\beta = 50\%$, realised precision is 27% (Big-Vul project), 19% (Big-Vul temporal) and 23% (Big-Vul→DiverseVul), and the DiverseVul-trained detectors certify nothing. Learned detectors can *clear* some code with a quantifiable risk, but at the precision available today they cannot be used to *confirm* vulnerabilities for more than a few per cent of the code.

5.4. RQ4: Accountability, robustness and explanations

The marginal guarantee is very uneven across function sizes. Table 9 and Figure 8 show that, even on exchangeable splits where the marginal guarantee holds, the miss rate at $\alpha = 10\%$ is 41.4% for the shortest third of DiverseVul functions and 2.2% for the longest third (25.8% versus 2.9% in Big-Vul). Short functions that contain vulnerabilities look like the many short benign functions, so most of the budget is consumed there. Length-conditional (Mondrian) calibration (Proposition 4) equalises the three groups (10.8/12.0/9.4% in DiverseVul and 8.0/10.5/8.2% in Big-Vul) at the price of a lower overall SAR (55.2 $\rightarrow$ 44.2% and 54.2 $\rightarrow$ 45.6%). Under shift it narrows the gap (e.g. Big-Vul project: 85.8/53.8/10.4% $\rightarrow$ 47.3/45.4/18.1%) but does not restore validity, as expected, since exchangeability remains violated.

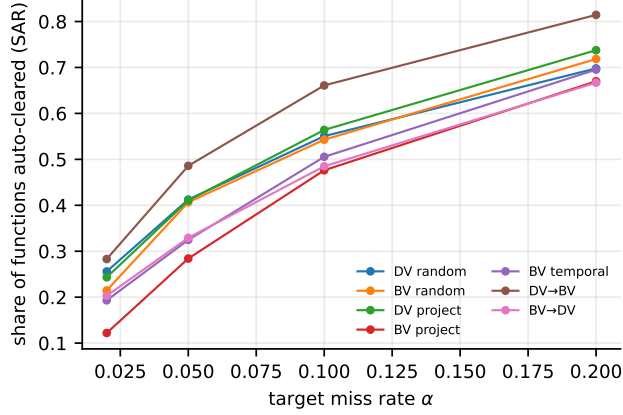


Figure 7: Share of functions that can be auto-cleared (SAR) versus nominal budget α for source-calibrated clearance (LR+LGBM). Values for shifted scenarios are *not* valid; see Table 7.

Per-CWE disparity. Figure 9 reports miss rates per CWE for the classes with at least 15 vulnerable test functions. In DiverseVul, LightGBM misses 17.6% of CWE-416 (use after free), 14.6% of CWE-476 (NULL dereference) and 11.3% of CWE-200, against 7.3% for CWE-119, 6.8% for CWE-125 and 9.1% for CWE-787 (all at a nominal 10%); in Big-Vul the range among the most frequent classes is 2.6% (CWE-125) to 14.9% (CWE-264). The budget is therefore met on average but not per vulnerability class; classes that are reported separately (for audits, for example) need their own calibration, which requires sufficient labelled examples per class.

Decision consistency under semantics-preserving rewrites. Table 10 and Figure 10 show how the clear/review decision changes when functions are rewritten without changing behaviour. Renaming local identifiers flips 10.4% of all decisions on average (and 5.6% of the vulnerable functions) and inserting dead code 6.0% (2.6%); both together flip 13.5% (6.7%). The score ranking is largely preserved (Spearman $\rho = 0.88$ – 0.997) and AUROC drops only by about 0.02, but the miss rate moves from 8.6–11.3% (original) to up to 13.5% after renaming (LightGBM, DiverseVul), i.e. the nominal guarantee does not survive cosmetic changes to the inputs. Because the triage threshold sits in the dense part of the score distribution, small score changes translate into many decision flips, which matters for accountability: the same code can be cleared or sent to review depending on variable names.

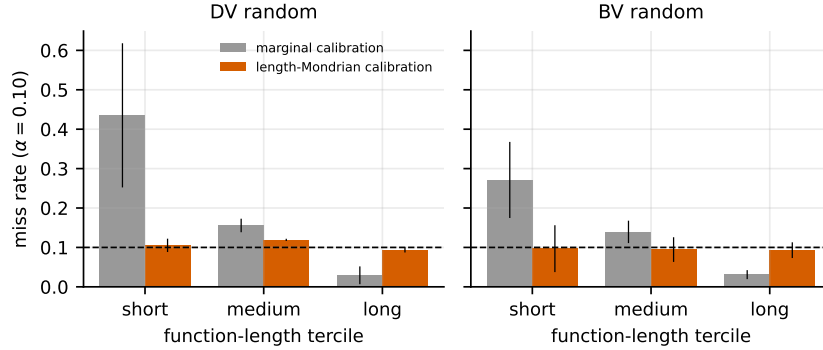


Figure 8: Miss rate per function-length tercile at $\alpha = 10\%$ on random splits, marginal versus Mondrian calibration (LR and LightGBM scorers, mean $\pm$ s.d.).

Explanations. For the explanation analysis we use an interpretable LightGBM on the 71 named code metrics (Table 11; AUROC 0.75 on both random splits and 0.69–0.71 on leak-free cross-dataset transfer, below the token-based scorers). TreeSHAP attributions are *faithful* by deletion (Figure 11, left): neutralising the ten highest-attributed features lowers the logit of the 10% highest-scored functions by 1.63 (DiverseVul) and 1.40 (Big-Vul), versus 1.09 and 0.67 for the top-ten features by global gain and -0.46 and -0.39 for random features, so local attributions identify the features that drive the decision better than a global ranking does. They are also *consistent*: the Spearman correlation of global importances is 0.97 between training seeds and 0.90 between a model trained on DiverseVul and one trained on Big-Vul. What they reveal is unflattering, however: the most important features are function size (characters, lines, tokens) and the number of parentheses (Figure 11, right), i.e. the model largely predicts that longer, more complex functions are vulnerable. This is consistent with the length-dependent miss rate above and with the known confound that functions touched by security fixes tend to be larger. As a check on identifier shortcuts in the token model, we compared the 200 highest positive-weight lexical features of the DiverseVul logistic regression with frequency-matched random features: the former occur in a median of 11 projects versus 13.5 for the controls (6% versus 3% occur in at most five projects), i.e. we do not find that the highest-weighted tokens are markedly more project-specific.

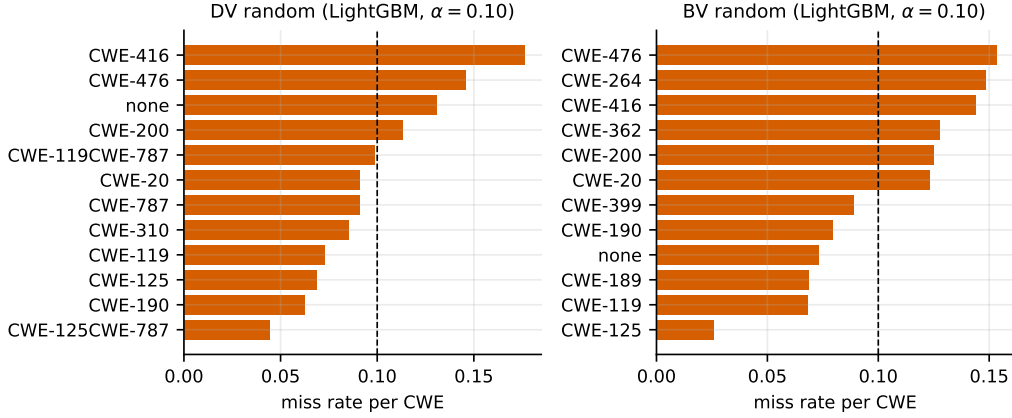


Figure 9: Per-CWE miss rate at $\alpha = 10\%$ on random splits (LightGBM, mean over seeds); the dashed line is the nominal budget.

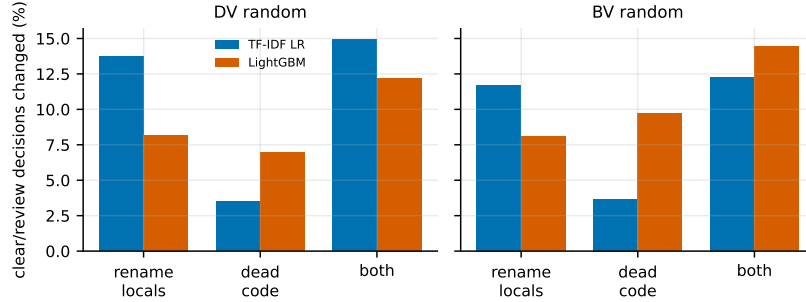


Figure 10: Share of clear/review decisions that change after each rewrite (5 200 test functions per scenario, $\alpha = 10\%$).

5.5. RQ5: Operational value

Comparison with a rule-based analyser. Flawfinder reports at least one hit for only 4.7–17.5% of the functions and its scores barely rank the vulnerable functions (AUROC 0.535–0.611, Table 12). Its only conservative policy, clearing functions without hits, auto-clears 82.5–95.3% of the code but misses 62.9–89.8% of the vulnerabilities. At the same clearance share the learned scorer misses 49.6–81.3%, and for the same (high) miss rate it could clear 84.7–98.8% (Figure 12). Neither is useful as an unattended filter at those operating points; the contrast is that the learned scorer provides a whole curve from which a budget such as 10% can be chosen and certified, whereas

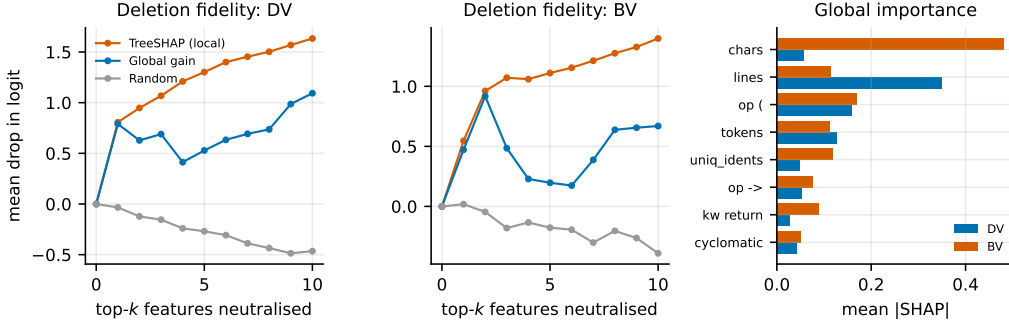


Figure 11: Explanation analysis for the metrics-only LightGBM. Left and centre: deletion fidelity (mean logit drop when the top- k features of each function are neutralised). Right: mean $|\text{SHAP}|$ of the eight most important features.

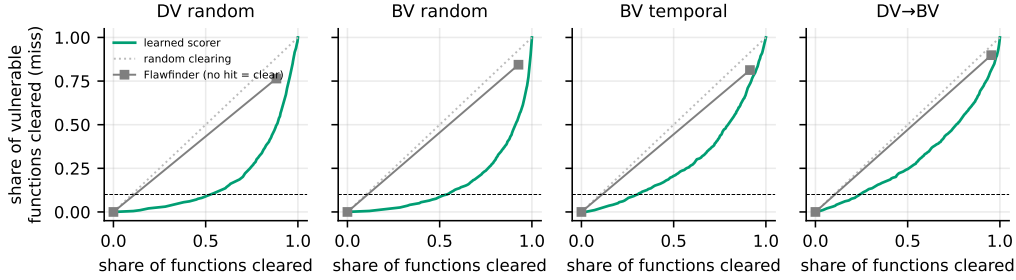


Figure 12: Miss rate versus share of functions cleared for the learned scorer (curve) and Flawfinder (square; “no hit = clear”). Dashed: 10% budget; dotted: random clearing.

Flawfinder provides a single, uncontrolled point.

Economic view. Table 13 and Figure 13 apply the cost model of Eq. (6) with α^* chosen on ID calibration data. On random splits and for a missed vulnerability that costs 50 reviews, the optimal budget is $\alpha^* = 0.07\text{--}0.09$, the rule clears 48–53% of the code, and expected cost is 0.68–0.73 of reviewing everything. Under project, temporal and cross-dataset shift the same procedure yields costs of 0.95–1.27, i.e. no saving or a loss, because the miss rate it believes to have ($\approx \alpha^*$) is far from the realised one. For $c_m = 10$, triage appears to save 46–69% of the cost but misses 35–75% of the vulnerabilities, and for $c_m = 200$ the saving is only 4–6% on random splits and negative under shift; for $c_m = 1000$ “review everything” is optimal in all scenarios. In short: delegation pays when a miss is worth tens, not hundreds, of reviews, and only

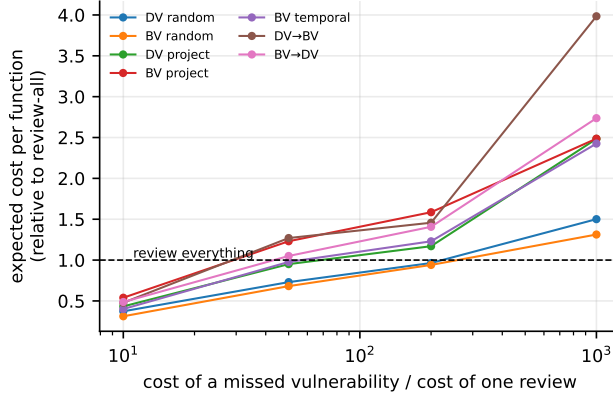


Figure 13: Expected cost per function relative to reviewing everything as a function of the cost ratio c_m (LR+LGBM, α^* selected on ID calibration data).

if the miss rate is calibrated on the deployment distribution.

5.6. RQ6: Pre-trained code model and shortcut learning

Tables 14 and 16 and Figure 14 report the CodeBERT experiment on the 30 000-function subsample (Section 4.5).

Raw text rewards a dataset artifact. With raw text as input, CodeBERT looks excellent on Big-Vul: the CodeBERT ensemble reaches AUROC 0.893 (random), 0.828 (project) and 0.844 (temporal), against 0.825, 0.709 and 0.728 for the TF-IDF+LightGBM scorer trained on the same subsample. The formatting-only classifier does even better (0.924, 0.895, 0.909; AUPRC 0.59–0.65 against 0.12–0.25 for the lexical scorers), and the advantage does not transfer: on DiverseVul the formatting-only classifier reaches 0.730 (random) and 0.690 (project), and Big-Vul→DiverseVul falls to 0.627 (Table 14). The artifact is visible in the data (Table 15): 3.0% of the lines of vulnerable Big-Vul functions end in whitespace against 0.1% of benign ones, 24.3% versus 6.5% of the functions start with whitespace, and vulnerable functions contain 2.9 comment markers on average against 0.8. It is much weaker in DiverseVul, which has other, smaller differences (comment markers 5.3 versus 1.8). When the same CodeBERT is given normalised text, its Big-Vul AUROC drops by 0.12 in all three scenarios (0.774, 0.706, 0.721) while DiverseVul is unchanged (0.753 and 0.702 versus 0.745 and 0.698).

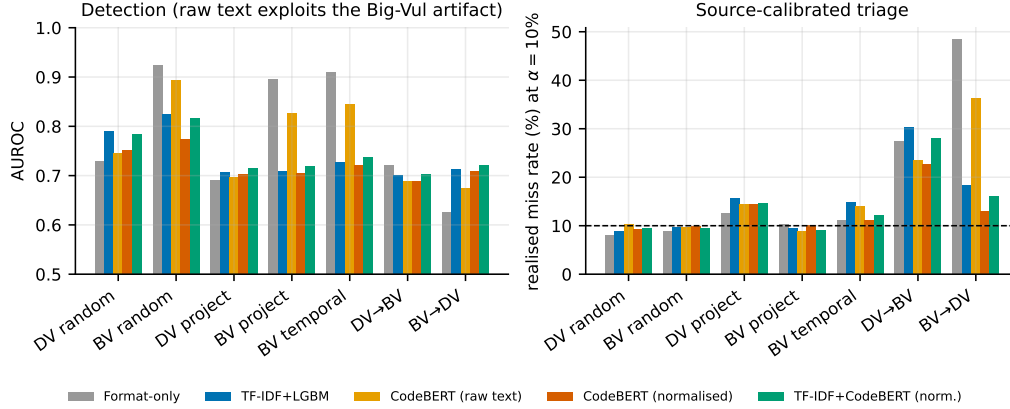


Figure 14: Left: AUROC of the formatting-only probe, the TF-IDF+LightGBM scorer and the CodeBERT scorers (raw and normalised text) on the subsample. Right: realised miss rate at $\alpha = 10\%$ when calibrated on source data (dashed: nominal).

Frozen CodeBERT adds little on normalised text. Without the artifact, the CodeBERT ensemble is below the lexical scorer on random splits (0.753 vs. 0.790 on DiverseVul, 0.774 vs. 0.825 on Big-Vul) and level under shift (0.689–0.721 vs. 0.701–0.728). Combining the two (TF-IDF+CodeBERT) is the best normalised scorer in every shifted scenario, but by only 0.002–0.010 AUROC (AUPRC 0.123–0.154 vs. 0.115–0.148). Frozen general-purpose embeddings thus do not change the picture of Section 5.1; fine-tuned encoders could do better, which we could not test (Section 7).

The guarantee findings replicate. For every CodeBERT-based scorer the realised miss rate at $\alpha = 10\%$ is on target on random splits (CodeBERT ensemble 9.6% normalised, 10.0% raw; TF-IDF 9.2%; combined 9.5%) and exceeded under shift (averages over the five shifted scenarios: 14.3% normalised CodeBERT, 19.4% raw CodeBERT, 17.7% TF-IDF, 16.0% combined; Table 16), and all 504 (normalised) and 441 (raw) observations satisfy the KS bound of Proposition 3. Calibrating on 100 labelled target positives restores the budget for every scorer (mean miss 9.2–9.6%, range 7.5–10.1% over scenarios), and the PAC variant keeps the violation frequency at or below 1.3%. The excess under shift is smaller than in the main study (1.4–1.9 $\times$ against 2.7 $\times$) because of the smaller training sets and, above all, because the project partitions of the subsample are milder (see Section 5.3); the cross-dataset violation remains large (22.7–30.4% for DiverseVul→Big-Vul).

A scorer can be valid and efficient for the wrong reason. The shortcut scorers are the most efficient. Calibrated on 100 target positives, the formatting-only probe clears 62.4% and 54.5% of Big-Vul functions under project and temporal shift, and raw-text CodeBERT 53.8% and 53.7%, against 27–32% for the lexical scorers and 26.9 and 26.1% for normalised CodeBERT; within Big-Vul the formatting-only probe meets its budget with source calibration (miss 10.2% and 11.1% under project and temporal shift) and raw CodeBERT nearly so (8.9% and 14.0%). The same scorers break the moment the shortcut does not hold: calibrated on Big-Vul and applied to DiverseVul they miss 48.4% (formatting-only) and 36.2% (raw CodeBERT) of the vulnerabilities at a 10% budget, against 13.0% for normalised CodeBERT and 18.3% for the lexical scorer. A conformal guarantee is a statement about the calibration distribution; it says nothing about whether the score separates vulnerabilities for a reason that survives deployment. This is one more reason to audit inputs for shortcuts and to calibrate on data from the deployment distribution.

6. Discussion

What the results mean for practice. The central message is operational: a detector’s usefulness as a *filter* is a property of the pair (detector, calibration data), not of the detector alone. Our findings translate into a short deployment recipe. (1) *Never calibrate the miss budget on a convenient hold-out of the training distribution when the tool is going to be used on other projects or later releases.* The resulting number is not conservative or optimistic by a small margin; it was off by a factor of 2.7 on average, and by 3.7 in the worst case. (2) *Budget for labelled data from the deployment distribution.* About 100 labelled vulnerable functions from the target (the new project, or the latest release window) are enough to recover the nominal miss rate on average; 25–50 are usable with a conservative finite-sample correction; with fewer than $\lceil 1/\alpha \rceil - 1$ labelled positives no clearance is possible at all. (3) *Choose between the expected and the high-probability guarantee.* The expected guarantee is exact but a single calibration draw exceeds the budget by two points in 16–68% of the draws at $k = 100$; the PAC rule brings this to $\leq 7\%$ for roughly 30% less automation. (4) *Calibrate per group when the group is observable.* Function length was an easy and effective stratifier; per-class calibration would require more labelled data than most teams have. (5) *Monitor.* Every cleared function that later turns out to be vulnerable is a labelled event; Algorithm 1 re-calibrates on such events. (6) *Check the*

economics. If a missed vulnerability is worth hundreds of reviews, delegating a share of the code to a detector of this quality does not pay off; if it is worth tens, the saving of 27–32% at $c_m = 50$ on exchangeable data turns into a change of between +5% and −27% (a loss) under shift when the budget is calibrated on source data.

Shortcuts make the guarantee a statement about the data, not the model. The CodeBERT experiment shows that a scorer can satisfy its miss budget within a dataset and clear more than half of the code because it reads formatting rather than vulnerability-related content, and then miss 36–48% of the vulnerabilities on another dataset. The practical lesson is twofold: audit inputs for shortcuts (a formatting-only probe costs minutes) and calibrate on data from the deployment distribution; a high clearance rate that coincides with a surprisingly easy detection task is a warning sign, not a success. For research it means that results on Big-Vul obtained from raw text deserve a re-check against normalised text.

What the results mean for research. We suggest that evaluations of vulnerability detectors report, besides AUROC/F1, the *miss-versus-clearance* curve at fixed budgets on shift-aware splits, together with a leakage audit. AUROC correlated only weakly with the clearable share at $\alpha = 5\%$ ($\rho = 0.38$) and the optimism of in-distribution validation reached a factor of 3.8 for AUPRC. The audit also shows that cross-dataset claims between these two corpora must remove shared functions: otherwise the reported AUPRC is $3.5\text{--}5.6\times$ too high. Finally, the negative result for covariate-shift weighting is informative: weighting is the standard shift remedy in the conformal literature, and for source code its assumptions (a stable conditional $\Pr(Y|x)$ and a well-estimated density ratio in a very high-dimensional token space) do not hold well enough in practice.

Relation to trustworthy automated software engineering. The special-issue themes of transparency, accountability and governance appear here as concrete, testable artefacts: the miss budget is an auditable statement ("at most α of the vulnerabilities in the calibration distribution are cleared"), its failure under shift is detectable once any labelled outcomes exist, the unevenness across sizes and CWEs shows where the statement hides risk, and the consistency study shows that the decision, not only the score, must be assessed under benign input changes. The approach transfers to other automated decisions in the development pipeline, e.g. filtering of software-composition-analysis

alerts, triage of AI-generated code for review, or deciding which dependencies need manual vetting, wherever a rare, costly event is missed silently and labelled outcomes arrive over time.

Limitations that shape how to read the results. The size dependence found by the explanation analysis suggests that part of the predictive signal in these corpora is a size confound rather than vulnerability semantics, which is a data issue and not a triage issue, but it limits what any detector trained on them can deliver. Stronger detectors would raise the achievable automation but would not change the logic of Propositions 1 and 3. Online variants that adapt the threshold over time are a natural extension for drift that is gradual rather than abrupt.

7. Threats to validity

Construct validity. Labels follow the datasets’ convention: a function is vulnerable if it was modified by a vulnerability-fixing change, and benign otherwise. Benign functions may contain undiscovered flaws, and vulnerable ones may be mislabelled; label noise in such corpora is documented Croft et al. (2023). Our “miss rate” is therefore the miss rate with respect to these labels, and the true miss rate in deployment could be higher; the guarantee is only as meaningful as the calibration labels, which is why the monitoring step of Algorithm 1 matters. The publication year of a CVE is a proxy for the age of the code, and the CWE annotation is incomplete, which limits the per-CWE analysis to classes with at least 15 vulnerable test functions. We remove exact duplicates after normalisation but not near-duplicates (clones with renamed identifiers or minor edits), so random-split numbers are probably still optimistic; this makes the contrast with the shifted scenarios conservative. The semantics-preserving rewrites are implemented with lexical analysis rather than a compiler front-end; renaming is restricted to declared locals and parameters and was not validated by recompilation.

Internal validity. Hyper-parameters are fixed across scenarios instead of being tuned per scenario, so absolute detection scores are lower than what careful tuning might reach; the triage conclusions concern the *relationship* between nominal and realised miss rates, which does not depend on the absolute score. The realised miss rates are estimated on 600–2 600 vulnerable test functions per scenario, giving a sampling error of roughly one percentage point at $\alpha = 10\%$; we use a two-point tolerance for violation counts and

report standard deviations over three seeds. The conformal implementation is checked by a unit test against the p -value definition and by simulation of the finite-sample guarantee. The domain classifier behind the weighted variant is trained on the same features as the detector, and a better density-ratio estimator could do better, so the negative result for weighting should be read as “not effective with standard estimators” rather than “never effective”.

External validity. We study two C/C++ corpora, which share provenance (a substantial share of Big-Vul functions is also in DiverseVul, Table 1), so the cross-dataset scenario is a harder test than a random split but not an independent one. Due to the CPU-only environment we evaluate four lightweight detectors on the full corpora, and CodeBERT only as a frozen encoder (first 256 tokens, on a 30 000-function stratified subsample), rather than fine-tuned pre-trained code models or large language models; fine-tuned models usually reach higher AUROC, and truncation to 256 tokens discards the tail of the 46% of functions that are longer. We expect the logic of our findings to hold, as the CodeBERT experiment suggests, but the conformal guarantee depends on exchangeability and not on the scorer, and the *direction* of our findings under shift (guarantees that hold under exchangeability and break under shift) follows from Proposition 3. The *magnitude* of the violation and the size of the cost-saving may differ for stronger models and should be re-measured; this is the most important item of future work. Results on other languages, on repository-level detection and on industrial data may differ.

The formatting shortcut we report was found in the Hugging Face release of Big-Vul used here; we did not trace whether it originates in the original data collection or in the release, and other releases may differ. The formatting-only probe is an upper-bound check, not an explanation of every model’s behaviour.

Conclusion validity. Blocks (scenario–seed pairs) are not fully independent because scenarios share data, so p -values are indicative; we therefore also report effect sizes and the full tables. The cost analysis depends on the assumed cost ratio c_m , which we vary over two orders of magnitude instead of picking one.

8. Conclusion

We asked when an automated vulnerability detector can be trusted to clear code. Framing clearance as risk control gives a precise answer: it can

be trusted to the extent that the miss rate was calibrated on data that resembles the code it will see. On exchangeable splits conformal clearance delivers its guarantee on real detector scores, clearing 54–55% of functions at a 10% miss budget. Under project, temporal and cross-dataset shift the same procedure misses $2.7\times$ the nominal share on average, covariate-shift weighting does not repair it, and about 100 labelled target positives do, at roughly half of the automation. The guarantee is also uneven across function sizes and vulnerability classes, sensitive to cosmetic code changes, and economically attractive only when a miss costs tens of reviews. Frozen CodeBERT scorers replicate these findings, and expose a formatting shortcut in Big-Vul that makes raw-text models look valid and efficient within the dataset and unreliable beyond it. Together with a leakage audit that shows cross-dataset performance can be overstated by up to a factor of six, these results argue for reporting miss-versus-clearance curves on shift-aware splits and for treating the miss budget as a monitored, re-calibrated operational quantity.

Future work will repeat the study with fine-tuned code language models and large language models (we could only use frozen CodeBERT embeddings), evaluate adaptive (online) conformal variants and better density-ratio estimators for shift, extend the analysis to other C/C++-external languages and to repository-level detection, and apply the same risk-control formulation to other automation decisions in the software supply chain.

Data and code availability

All code, split definitions, cached detector scores and the scripts that generate every table and figure are in the repository that accompanies this manuscript. Both datasets are public (DiverseVul, Big-Vul) and were obtained from the Hugging Face hub.

References

Allamanis, M., 2019. The adverse effects of code duplication in machine learning models of code, in: Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!).

- Angelopoulos, A.N., Bates, S., 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv preprint arXiv:2107.07511 .
- Angelopoulos, A.N., Bates, S., Candès, E.J., Jordan, M.I., Lei, L., 2021. Learn then test: Calibrating predictive algorithms to achieve risk control. arXiv preprint arXiv:2110.01052 .
- Angelopoulos, A.N., Bates, S., Fisch, A., Lei, L., Schuster, T., 2022. Conformal risk control. arXiv preprint arXiv:2208.02814 .
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2022. Deep learning based vulnerability detection: Are we there yet? IEEE Transactions on Software Engineering 48, 3280–3296.
- Chen, Y., Ding, Z., Alowain, L., Chen, X., Wagner, D., 2023. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID).
- Croft, R., Babar, M.A., Kholoosi, M.M., 2023. Data quality for software vulnerability datasets, in: Proceedings of the 45th International Conference on Software Engineering (ICSE).
- Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y., 2025. Vulnerability detection with code language models: How far are we?, in: Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE).
- Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories (MSR).
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020.
- Geifman, Y., El-Yaniv, R., 2017. Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems.

- Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML).
- Johnson, B., Song, Y., Murphy-Hill, E., Bowdidge, R., 2013. Why don't software developers use static analysis tools to find bugs?, in: Proceedings of the 35th International Conference on Software Engineering (ICSE).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y., 2017. LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems.
- Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., Lee, S.I., 2020. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence* 2, 56–67.
- Lundberg, S.M., Lee, S.I., 2017. A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems.
- Luo, R., Zhao, S., Kuck, J., Ivanovic, B., Savarese, S., Schmerling, E., Pavone, M., 2022. Sample-efficient safety assurances using conformal prediction, in: International Workshop on the Algorithmic Foundations of Robotics (WAFR).
- Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L., 2019. TESSERACT: Eliminating experimental bias in malware classification across space and time, in: 28th USENIX Security Symposium.
- Rabin, M.R.I., Bui, N.D.Q., Wang, K., Yu, Y., Jiang, L., Alipour, M.A., 2021. On the generalizability of neural program models with respect to semantic-preserving program transformations.
- Shafer, G., Vovk, V., 2008. A tutorial on conformal prediction. *Journal of Machine Learning Research* 9, 371–421.
- Tibshirani, R.J., Foygel Barber, R., Candès, E., Ramdas, A., 2019. Conformal prediction under covariate shift, in: Advances in Neural Information Processing Systems.

- Vovk, V., Gammernan, A., Shafer, G., 2005. Algorithmic Learning in a Random World. Springer.
- Yefet, N., Alon, U., Yahav, E., 2020. Adversarial examples for models of code. Proceedings of the ACM on Programming Languages (OOPSLA) .
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Advances in Neural Information Processing Systems.

Table 4: Detection performance on the held-out test sets (mean $\pm$ s.d. over 3 seeds; best per row in bold). MCC uses the F1-optimal threshold chosen on ID calibration data.

Scenario	TF-IDF LR	TF-IDF SVM	LightGBM	LR+LGBM
<i>Panel A: AUPRC</i>				
DV random	0.229 $\pm$ 0.002	0.248 $\pm$ 0.003	0.231 $\pm$ 0.004	0.248 $\pm$ 0.005
BV random	0.427 $\pm$ 0.010	0.432 $\pm$ 0.012	0.337 $\pm$ 0.013	0.407 $\pm$ 0.015
DV project	0.120 $\pm$ 0.015	0.126 $\pm$ 0.018	0.139 $\pm$ 0.009	0.141 $\pm$ 0.013
BV project	0.142 $\pm$ 0.027	0.137 $\pm$ 0.025	0.143 $\pm$ 0.040	0.151 $\pm$ 0.037
BV temporal	0.103 $\pm$ 0.001	0.102 $\pm$ 0.000	0.110 $\pm$ 0.005	0.113 $\pm$ 0.003
DV $\rightarrow$ BV	0.151 $\pm$ 0.002	0.162 $\pm$ 0.001	0.163 $\pm$ 0.000	0.171 $\pm$ 0.001
BV $\rightarrow$ DV	0.137 $\pm$ 0.001	0.139 $\pm$ 0.001	0.136 $\pm$ 0.002	0.144 $\pm$ 0.002
<i>Panel B: AUROC</i>				
DV random	0.832 $\pm$ 0.001	0.832 $\pm$ 0.002	0.800 $\pm$ 0.002	0.836 $\pm$ 0.002
BV random	0.869 $\pm$ 0.013	0.867 $\pm$ 0.012	0.830 $\pm$ 0.010	0.869 $\pm$ 0.012
DV project	0.704 $\pm$ 0.030	0.706 $\pm$ 0.033	0.742 $\pm$ 0.022	0.740 $\pm$ 0.026
BV project	0.683 $\pm$ 0.012	0.674 $\pm$ 0.012	0.684 $\pm$ 0.006	0.694 $\pm$ 0.005
BV temporal	0.710 $\pm$ 0.002	0.701 $\pm$ 0.001	0.710 $\pm$ 0.013	0.722 $\pm$ 0.006
DV $\rightarrow$ BV	0.679 $\pm$ 0.001	0.685 $\pm$ 0.001	0.711 $\pm$ 0.003	0.710 $\pm$ 0.002
BV $\rightarrow$ DV	0.711 $\pm$ 0.001	0.704 $\pm$ 0.002	0.709 $\pm$ 0.006	0.721 $\pm$ 0.003
<i>Panel C: MCC</i>				
DV random	0.292 $\pm$ 0.004	0.291 $\pm$ 0.006	0.257 $\pm$ 0.002	0.288 $\pm$ 0.006
BV random	0.415 $\pm$ 0.013	0.413 $\pm$ 0.018	0.319 $\pm$ 0.011	0.387 $\pm$ 0.014
DV project	0.120 $\pm$ 0.025	0.134 $\pm$ 0.024	0.163 $\pm$ 0.030	0.165 $\pm$ 0.030
BV project	0.112 $\pm$ 0.032	0.109 $\pm$ 0.036	0.125 $\pm$ 0.052	0.133 $\pm$ 0.057
BV temporal	0.071 $\pm$ 0.011	0.073 $\pm$ 0.008	0.119 $\pm$ 0.007	0.121 $\pm$ 0.016
DV $\rightarrow$ BV	0.150 $\pm$ 0.007	0.161 $\pm$ 0.004	0.158 $\pm$ 0.005	0.166 $\pm$ 0.002
BV $\rightarrow$ DV	0.138 $\pm$ 0.002	0.146 $\pm$ 0.007	0.140 $\pm$ 0.006	0.149 $\pm$ 0.007

Table 5: Expected calibration error (ECE, lower is better) on the test sets; Platt scaling and isotonic regression are fitted on ID calibration data. Mean over 3 seeds.

Scenario	LR raw	LR Platt	LR isotonic	LGBM raw	LGBM Platt	LGBM isotonic
DV random	0.032	0.015	0.006	0.018	0.006	0.004
BV random	0.015	0.004	0.004	0.008	0.005	0.004
DV project	0.026	0.010	0.017	0.009	0.005	0.008
BV project	0.037	0.022	0.023	0.025	0.022	0.023
BV temporal	0.022	0.015	0.016	0.016	0.016	0.016
DV $\rightarrow$ BV	0.031	0.020	0.021	0.008	0.018	0.018
BV $\rightarrow$ DV	0.035	0.023	0.026	0.022	0.022	0.023

Table 6: Realised miss rate (% , mean $\pm$ s.d. over seeds) of the conformal clearance rule calibrated on source-distribution (ID) data, for four nominal budgets α (LR+LGBM scorer).

Scenario	$\alpha=2\%$	$\alpha=5\%$	$\alpha=10\%$	$\alpha=20\%$
DV random	2.4 $\pm$ 0.2	5.5 $\pm$ 0.5	10.1 $\pm$ 0.6	19.8 $\pm$ 1.7
BV random	1.5 $\pm$ 1.2	4.6 $\pm$ 2.5	8.6 $\pm$ 2.3	18.2 $\pm$ 2.3
DV project	5.9 $\pm$ 1.6	13.8 $\pm$ 3.0	23.6 $\pm$ 3.8	40.5 $\pm$ 6.5
BV project	5.0 $\pm$ 1.6	13.5 $\pm$ 2.1	24.0 $\pm$ 7.6	39.8 $\pm$ 12.5
BV temporal	5.6 $\pm$ 1.1	11.5 $\pm$ 3.0	21.7 $\pm$ 2.2	37.4 $\pm$ 3.2
DV $\rightarrow$ BV	10.7 $\pm$ 0.6	22.4 $\pm$ 0.3	37.0 $\pm$ 0.3	55.3 $\pm$ 1.6
BV $\rightarrow$ DV	5.9 $\pm$ 1.6	11.2 $\pm$ 2.0	20.3 $\pm$ 1.9	35.8 $\pm$ 3.4

Table 7: Conformal clearance at $\alpha=10\%$ (LR+LGBM scorer; all figures in %). ID: calibrated on source data; Wtd: covariate-shift weighted; $k=100$: calibrated on 100 labelled target positives; PAC: $k=100$ with the high-probability rule ($\delta=0.1$) and its violation frequency $\Pr(M > \alpha + 2pp)$. A valid rule has miss $\lesssim 10$.

Scenario	ID miss	ID SAR	Wtd miss	Wtd SAR	$k=100$ miss	$k=100$ SAR	PAC miss	PAC SAR	PAC viol.
DV random	10.1 $\pm$ 0.6	55.1	—	—	—	—	—	—	—
BV random	8.6 $\pm$ 2.3	54.3	—	—	—	—	—	—	—
DV project	23.6 $\pm$ 3.8	56.4	15.1	41.1	9.2	31.8	5.4	22.4	2
BV project	24.0 $\pm$ 7.6	47.6	18.6	34.5	13.0	28.1	6.9	15.9	7
BV temporal	21.7 $\pm$ 2.2	50.5	22.6	51.4	9.7	28.5	6.0	20.1	0
DV $\rightarrow$ BV	37.0 $\pm$ 0.3	66.1	14.9	30.2	10.1	26.7	6.1	18.2	2
BV $\rightarrow$ DV	20.3 $\pm$ 1.9	48.5	19.5	46.6	10.0	30.0	5.9	20.2	2

Table 8: Certified auto-flagging (Learn-then-Test, $\delta=0.05$): share of test functions flagged (%) and realised precision (%); “—” = no threshold could be certified.

Scenario	$\beta=.3$ flagged	prec.	$\beta=.5$ flagged	prec.	$\beta=.7$ flagged	prec.
DV random	1.35	41	0.09	80	0.02	78
BV random	8.06	33	2.57	57	0.30	87
DV project	0.16	39	0.00	43	0.00	—
BV project	9.39	18	1.64	27	0.10	45
BV temporal	6.67	15	0.54	19	0.03	23
DV $\rightarrow$ BV	0.33	56	0.00	79	0.00	75
BV $\rightarrow$ DV	11.25	16	3.02	23	0.71	27

Table 9: Miss rate (%) per function-length tercile at $\alpha=10\%$ for marginal (left) and Mondrian (right) calibration, with the overall share cleared (SAR). LR+LGBM scorer, ID calibration.

Scenario	short	medium	long	SAR	short	medium	long	SAR
DV random	41.4	16.3	2.2	55.2	10.8	12.0	9.4	44.2
BV random	25.8	14.8	2.9	54.2	8.0	10.5	8.2	45.6
DV project	84.5	42.8	4.5	55.4	34.0	25.9	19.8	43.7
BV project	85.8	53.8	10.4	55.0	47.3	45.4	18.1	40.6
BV temporal	75.7	47.5	8.5	54.1	33.2	29.4	14.5	37.7
DV→BV	90.5	51.3	7.8	57.1	37.5	33.5	29.1	45.2
BV→DV	75.6	41.1	8.2	53.4	40.4	24.3	17.6	41.3

Table 10: Consistency of triage decisions ($\alpha=10\%$, ID calibration) on 1 200 vulnerable and 4 000 benign random test functions after semantics-preserving rewrites. “Flipped” = share of all (resp. vulnerable) functions whose clear/review decision changes.

Scenario	Model	Rewrite	AUROC	Miss (%)	Flipped (%)	Flipped, vuln. (%)	Spearman ρ
DV random	TF-IDF LR	original	0.838	9.4	0.0	0.0	1.000
DV random	TF-IDF LR	rename locals	0.824	10.5	13.8	5.4	0.898
DV random	TF-IDF LR	dead code	0.837	8.4	3.5	1.3	0.997
DV random	TF-IDF LR	both	0.823	8.9	14.9	5.7	0.893
DV random	LightGBM	original	0.804	11.2	0.0	0.0	1.000
DV random	LightGBM	rename locals	0.788	13.5	8.2	6.8	0.937
DV random	LightGBM	dead code	0.800	9.4	7.0	3.2	0.978
DV random	LightGBM	both	0.785	10.1	12.2	8.2	0.917
BV random	TF-IDF LR	original	0.859	8.6	0.0	0.0	1.000
BV random	TF-IDF LR	rename locals	0.848	8.4	11.7	4.7	0.924
BV random	TF-IDF LR	dead code	0.860	10.5	3.7	1.9	0.994
BV random	TF-IDF LR	both	0.849	10.0	12.3	5.6	0.917
BV random	LightGBM	original	0.825	8.9	0.0	0.0	1.000
BV random	LightGBM	rename locals	0.810	10.9	8.1	5.7	0.937
BV random	LightGBM	dead code	0.820	8.3	9.7	3.9	0.945
BV random	LightGBM	both	0.804	10.0	14.5	7.2	0.879

Table 11: Interpretable metrics-only LightGBM used for the explanation analysis (3 seeds); cross-dataset rows exclude leaked functions.

Train→test	AUROC	AUPRC
DV	0.749 ±0.002	0.164 ±0.003
BV	0.756 ±0.009	0.207 ±0.012
DV→BV	0.694 ±0.001	0.132 ±0.002
BV→DV	0.710 ±0.002	0.137 ±0.002

Table 12: Flawfinder (FF) versus the learned scorer (LR+LGBM rank average) on identical test functions (seed 0). The only conservative FF policy is to clear functions without any hit; its SAR and miss rate (%) are compared with the learned scorer’s miss rate at the same SAR and its SAR at the same miss rate.

Scenario	AUROC FF	AUROC ML	AUPRC FF	AUPRC ML	FF SAR	FF miss	ML miss @SAR	ML SAR @miss
DV random	0.570	0.830	0.075	0.230	88.3	76.4	49.6	95.4
BV random	0.548	0.851	0.065	0.380	92.7	84.4	54.4	98.8
DV project	0.568	0.709	0.070	0.126	90.6	79.7	71.8	94.3
BV project	0.611	0.700	0.101	0.141	82.5	62.9	58.9	84.7
BV temporal	0.555	0.726	0.058	0.110	91.4	81.3	73.3	94.7
DV→BV	0.535	0.697	0.066	0.163	95.3	89.8	81.3	98.4
BV→DV	0.570	0.707	0.079	0.141	88.3	77.2	68.1	92.7

Table 13: Cost-optimal miss budget α^* chosen on ID calibration data, its realised SAR (%) and expected cost per function relative to reviewing everything (= 1), for $c_m = 10, 50, 200$ review units per missed vulnerability. LR+LGBM scorer.

Scenario	α^*	SAR	cost	α^*	SAR	cost	α^*	SAR	cost
DV random	0.35	82	0.37	0.07	48	0.73	0.01	17	0.96
BV random	0.43	90	0.31	0.09	53	0.68	0.01	15	0.94
DV project	0.33	86	0.43	0.06	46	0.95	0.01	16	1.17
BV project	0.37	85	0.54	0.09	46	1.23	0.02	11	1.59
BV temporal	0.38	89	0.40	0.11	53	0.98	0.01	16	1.23
DV→BV	0.38	93	0.48	0.08	62	1.27	0.01	17	1.46
BV→DV	0.35	83	0.49	0.08	45	1.05	0.02	18	1.41

Table 14: Detection with frozen CodeBERT embeddings on the stratified subsample (mean over 3 seeds; AUPRC weighted to the true prevalence). Fmt: formatting-only classifier; CB raw / norm: CodeBERT on raw text / on comment-and-whitespace-normalised text; ensembles are standardised score sums.

Scenario	AUROC Fmt	AUROC TF-IDF	AUROC CB raw	AUROC CB norm	AUROC TF+CB	AUPRC Fmt	AUPRC TF-IDF	AUPRC CB raw	AUPRC CB norm	AUPRC TF+CB
DV random	0.730	0.790	0.745	0.753	0.784	0.143	0.208	0.168	0.167	0.203
BV random	0.924	0.825	0.893	0.774	0.816	0.651	0.251	0.445	0.180	0.240
DV project	0.690	0.708	0.698	0.702	0.715	0.122	0.137	0.122	0.135	0.147
BV project	0.895	0.709	0.828	0.706	0.719	0.588	0.142	0.312	0.142	0.152
BV temporal	0.909	0.728	0.844	0.721	0.737	0.647	0.115	0.291	0.121	0.123
DV→BV	0.720	0.701	0.688	0.689	0.703	0.193	0.148	0.126	0.139	0.154
BV→DV	0.627	0.712	0.674	0.709	0.721	0.101	0.147	0.113	0.140	0.153

Table 15: Formatting-shortcut audit on the stratified subsample (random 70/30 split). First two columns: LightGBM on 15 formatting features of the raw text only (indentation, trailing whitespace, comment markers, line lengths; AUPRC weighted to true prevalence). Next three: mean per function, vulnerable / benign. Last: AUROC of the main-study LightGBM with all metrics / with size-sensitive metrics (chars, lines, tokens) removed (seed 0).

Dataset	AUROC	AUPRC	Trailing-ws lines (%)	Starts with ws (%)	Comment markers	Main LGBM AUROC
Big-Vul	0.919	0.643	3.0 / 0.1	24.3 / 6.5	2.9 / 0.8	0.820 / 0.818
DiverseVul	0.721	0.145	0.5 / 0.2	4.2 / 7.9	5.3 / 1.8	0.801 / 0.802

Table 16: Triage with CodeBERT scorers at $\alpha=10\%$ (%). Left: realised miss rate when calibrated on source (ID) data; right: safe automation rate when calibrated on 100 labelled target positives (valid by Prop. 1). Subsample experiment; see Table 14 for column keys.

Scenario	miss Fmt	miss TF-IDF	miss CB raw	miss CB norm	miss TF+CB	SAR Fmt	SAR TF-IDF	SAR CB raw	SAR CB norm	SAR TF+CB
DV random	8.0	8.8	10.3	9.2	9.4	—	—	—	—	—
BV random	8.9	9.6	9.7	10.0	9.5	—	—	—	—	—
DV project	12.6	15.6	14.5	14.4	14.6	25.0	26.8	24.9	25.9	28.0
BV project	10.2	9.5	8.9	10.1	9.2	62.4	32.1	53.8	26.9	29.9
BV temporal	11.1	14.9	14.0	11.2	12.1	54.5	27.2	53.7	26.1	27.4
DV→BV	27.4	30.4	23.6	22.7	28.1	17.7	24.8	23.4	24.0	25.1
BV→DV	48.4	18.3	36.2	13.0	16.1	14.7	29.2	24.2	28.8	31.0
Mean, random	8.4	9.2	10.0	9.6	9.5	—	—	—	—	—
Mean, shifted	21.9	17.7	19.4	14.3	16.0	34.9	28.0	36.0	26.3	28.3